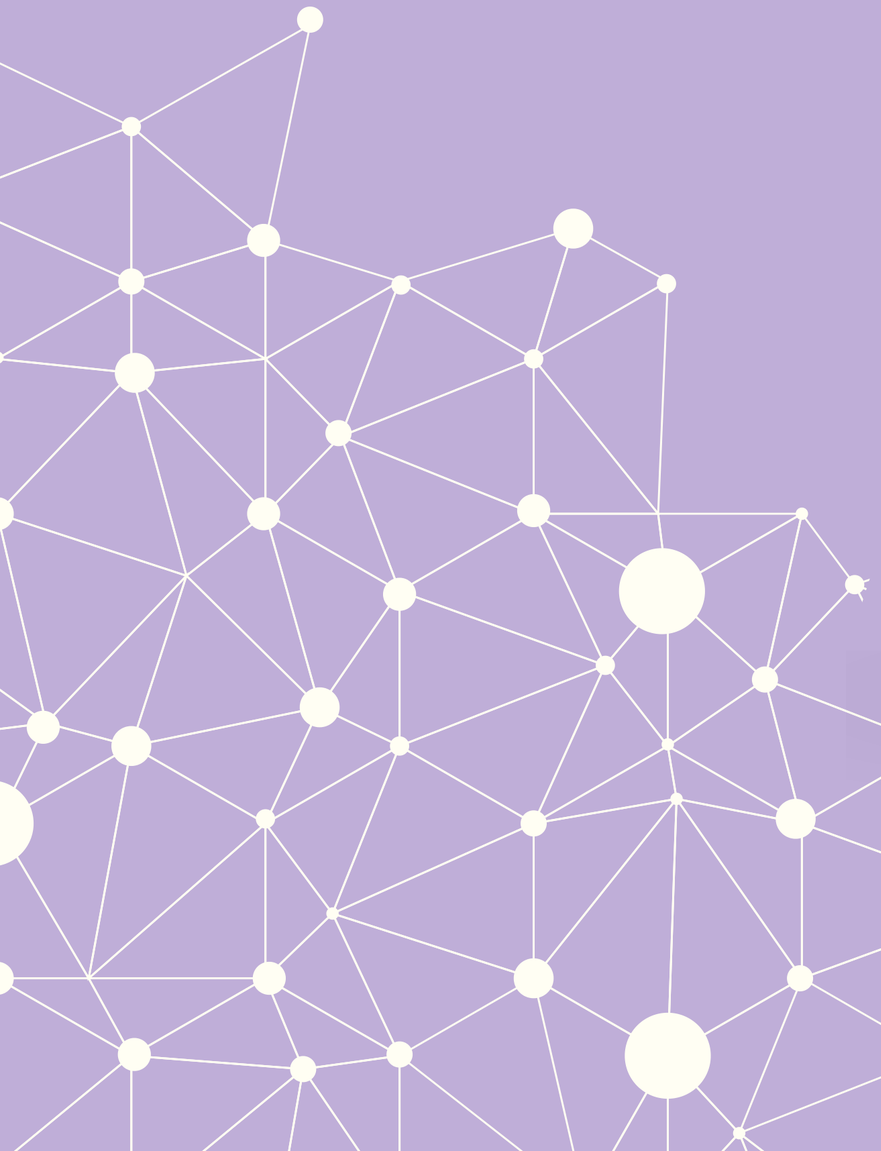
INSTITUTO FEDERAL DE BRAGANÇA PAULISTA



DOCKER IN

SENSIVAPP

Integrantes



ARTUR MIRANDA



CAMILA PEREIRA



GIOVANNA BRANDÃO

INTRODUÇÃO

> Pessoas com hipersensibilidade a estímulos visuais e sonoros e dificuldades de foco frequentemente encontram no ambiente diário desafios para sua qualidade de vida.

CONTEXTO

Domínio do Projeto

Saúde e bem-estar digital, com foco no apoio a pessoas com hipersensibilidade sensorial, dificuldades de foco e acompanhamento terapêutico.

Comunidade-alvo

Pessoas com hipersensibilidade sensorial e clinicas de profissionais da área da saúde que realizam acompanhamento terapêutico.



PROBLEMAS

Sobrecarga Sensorial Constante

O dia a dia é um campo de batalha sensorial com ruídos e luzes intensas, gerando exaustão.

Dificuldade de Foco e Desempenho

A sobrecarga sabota a concentração, impactando tarefas básicas e produtividade.

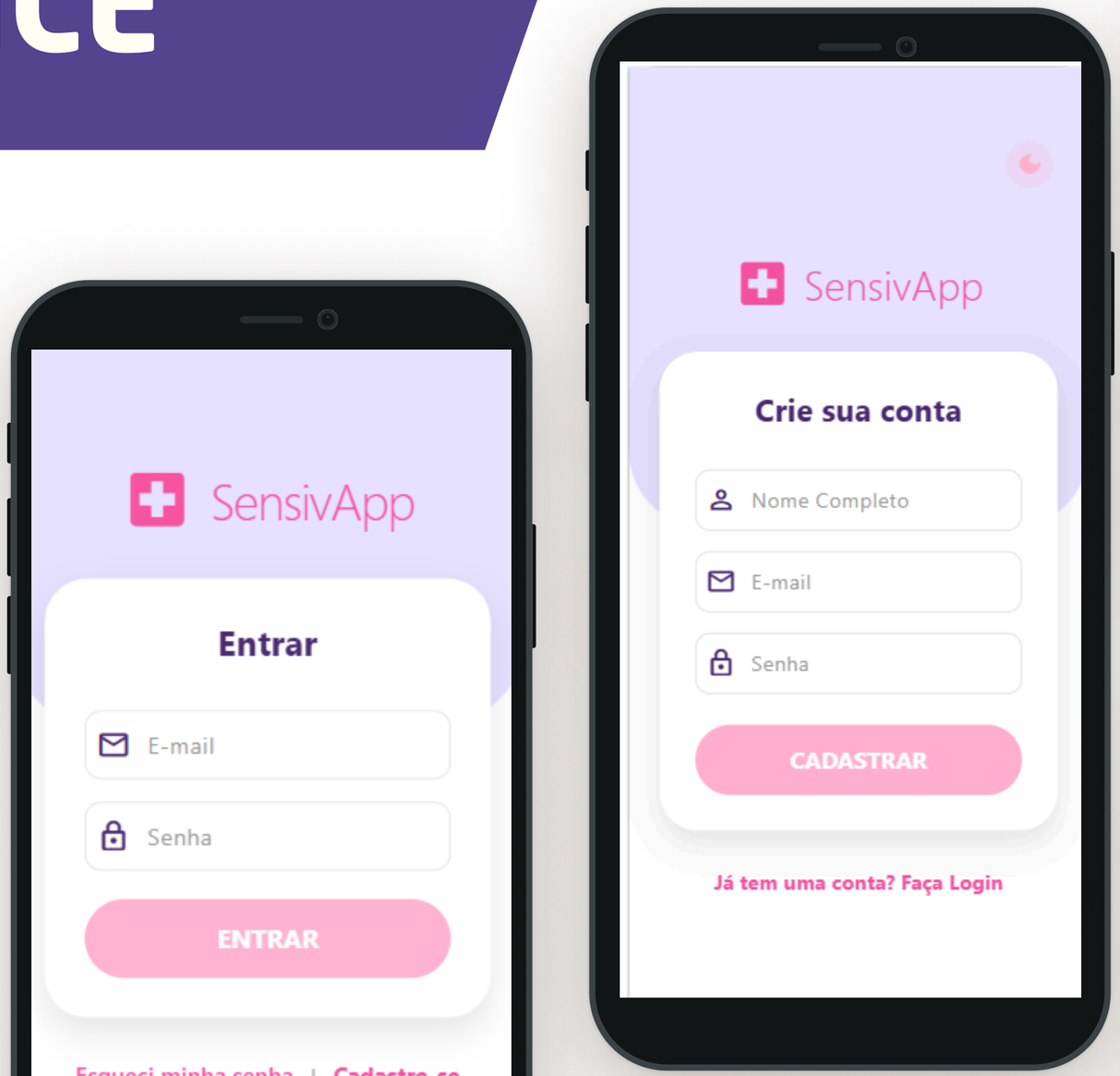
Limitação na Vida Social

A dificuldade em lidar com ambientes comuns limita a participação plena e a busca por bem-estar.

Estresse e Ansiedade Crônicos

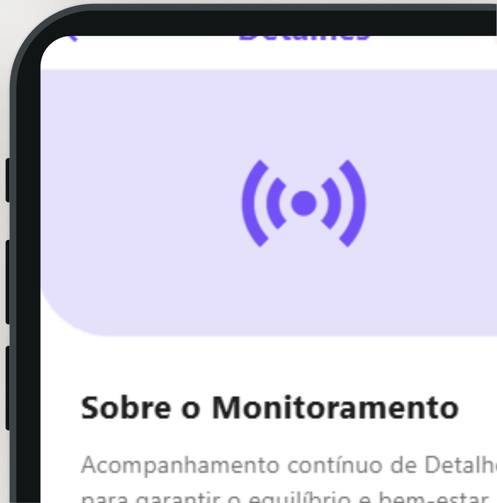
A exposição contínua a gatilhos leva a estresse crônico, frustração e crises de ansiedade.

INTERFACE

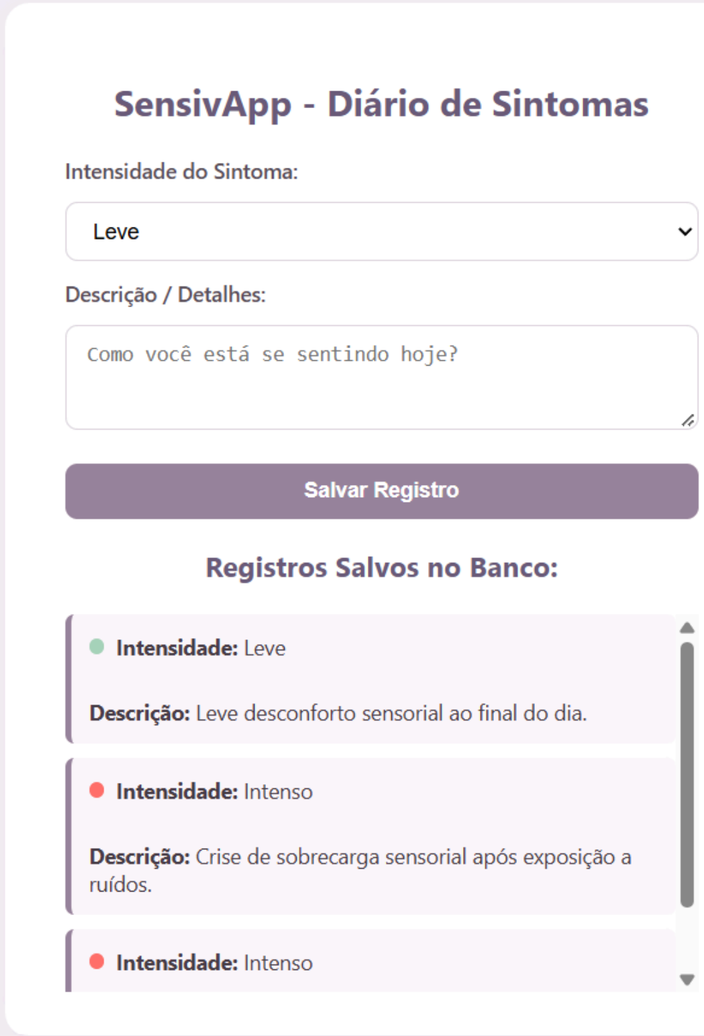

SensivApp
Entrar
E-mail
Senha
ENTRAR
Esqueci minha senha | Cadastre-se

SensivApp
Crie sua conta
Nome Completo
E-mail
Senha
CADASTRAR
Já tem uma conta? Faça Login


Ruído Externo
Dados em tempo real
Tarefas Médicas
Usar abafadores
Prescrição médica
Reduzir brilho da tela
Prescrição médica
Pausa em local calmo
Prescrição médica


Sobre o Monitoramento
Acompanhamento contínuo de Detalh

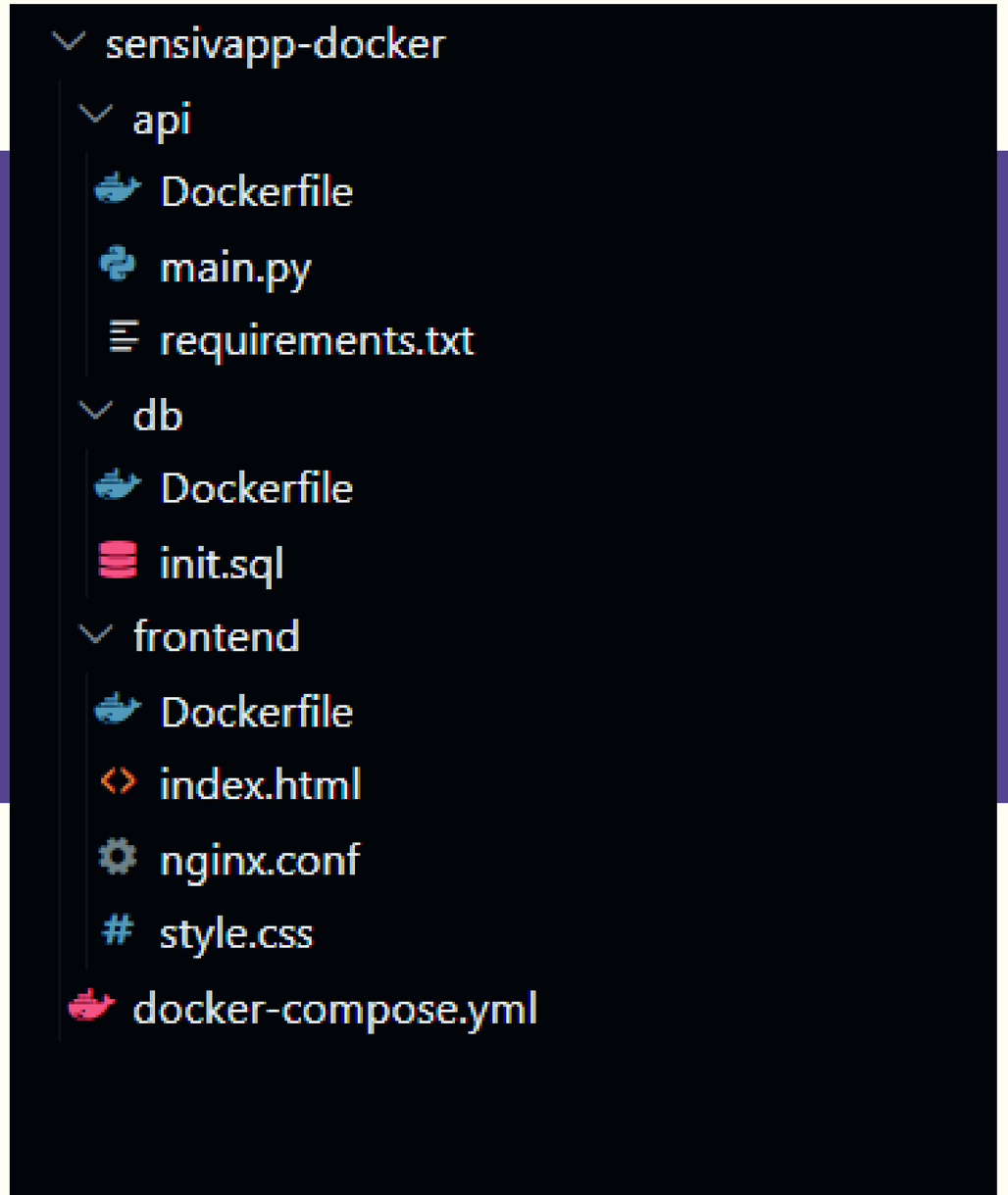
INTERFACE UTILIZADA



ESTRUTURA DE PASTAS

- **Modularidade:** Pastas independentes para Front, API e Banco.
- **Independência:** Builds isolados para cada componente.
- **Organização:** Clareza no versionamento e manutenção.

ESTRUTURA DE PASTAS



MICROSSERVIÇOS

Utilizamos microsserviços via Docker para isolar as camadas em containers independentes:

- Front-end com Nginx
- Back-end com FastAPI/Python
- Banco de Dados com PostgreSQL

Eles se comunicam de forma segura através de uma rede overlay e são orquestrados nativamente pelo Docker Swarm, garantindo alta disponibilidade, balanceamento de carga automático e persistência de dados.

docker-compose.yml

O QUE FAZ

Define toda a arquitetura dos microsserviços do SensivApp em formato de stack para o Docker Swarm, configurando o deploy dos nossos microsserviços.

ORQUESTRAÇÃO DE MICROSSERVIÇOS E RÉPLICAS

docker-compose.yml

```yaml
version: '3.8'

services:
  db:
    build: ./db
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: sensivdb
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - sensiv_net
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure

  api:
    image: sensiv-api:latest
    environment:
      - DATABASE_URL=postgresql://user:password@db:5432/sensivdb
    networks:
      - sensiv_net
    depends_on:
      - db
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure
```

ORQUESTRAÇÃO DE MICROSSERVIÇOS E RÉPLICAS

docker-compose.yml

```yaml
  frontend:
    image: sensiv-frontend:latest
    networks:
      - sensiv_net
    depends_on:
      - api
    ports:
      - "3000:80"
    deploy:
      replicas: 2
      restart_policy:
        condition: on-failure

  visualizer:
    image: dockersamples/visualizer:stable
    ports:
      - "8080:8080"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    networks:
      - sensiv_net
    deploy:
      placement:
        constraints: [node.role == manager]

volumes:
  pgdata:
```



ORQUESTRAÇÃO DE MICROSSERVIÇOS E RÉPLICAS

docker-
compose.yml

```yaml
networks:
  sensiv_net:
    driver: overlay
```

INFRAESTRUTURA DO BANCO DE DADOS

O BANCO DE DADOS (POSTGRESQL COMPLETO)

Cria a estrutura relacional do sistema e insere dados iniciais de teste, rodando o script de forma automatica quando o container do PostgreSQL sobe pela primeira vez.

DOCKERFILE

```
1    FROM postgres:15-alpine
2
3    COPY init.sql /docker-entrypoint-initdb.d/
4
5    EXPOSE 5432
```

BANCO DE DADOS RELACIONAL

```sql
CREATE TABLE IF NOT EXISTS usuarios (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    tipo_perfil VARCHAR(50) DEFAULT 'Paciente',
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS categorias_gatilhos (
    id SERIAL PRIMARY KEY,
    nome_gatilho VARCHAR(100) NOT NULL,
    descricao_gatilho TEXT
);

CREATE TABLE IF NOT EXISTS sintomas (
    id SERIAL PRIMARY KEY,
    usuario_id INT REFERENCES usuarios(id) ON DELETE CASCADE,
    gatilho_id INT REFERENCES categorias_gatilhos(id) ON DELETE SET NULL,
    intensidade VARCHAR(50) NOT NULL,
    descricao TEXT,
    data_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS configuracoes_usuario (
    id SERIAL PRIMARY KEY,
    usuario_id INT REFERENCES usuarios(id) ON DELETE CASCADE,
    modo_alto_contraste BOOLEAN DEFAULT FALSE,
    notificacoes_ativas BOOLEAN DEFAULT TRUE,
    limite_alerta_intensidade VARCHAR(50) DEFAULT 'Intenso'
);
```

BANCO DE DADOS RELACIONAL

```sql
CREATE TABLE IF NOT EXISTS notificacoes (
    id SERIAL PRIMARY KEY,
    usuario_id INT REFERENCES usuarios(id) ON DELETE CASCADE,
    mensagem TEXT NOT NULL,
    lida BOOLEAN DEFAULT FALSE,
    enviada_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS logs_auditoria (
    id SERIAL PRIMARY KEY,
    acao VARCHAR(100) NOT NULL,
    detalhes TEXT,
    nivel_severidade VARCHAR(20) DEFAULT 'INFO',
    data_acao TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO usuarios (nome, email, tipo_perfil) VALUES
('Giovanna Brandão', 'giovanna@sensivapp.com', 'Paciente'),
('Camila Pereira', 'camila@sensivapp.com', 'Cuidador');

INSERT INTO categorias_gatilhos (nome_gatilho, descricao_gatilho) VALUES
('Ruído Excessivo', 'Sons altos, multidões ou frequências agudas.'),
('Luz Intensa', 'Luzes fluorescentes fortes ou claridade solar direta.'),
('Sobrecarga Visual', 'Ambientes muito poluídos visuais ou telas piscando.');

INSERT INTO sintomas (usuario_id, gatilho_id, intensidade, descricao) VALUES
(1, 1, 'Leve', 'Leve desconforto sensorial ao final do expediente de estudos.'),
(1, 2, 'Intenso', 'Crise de sobrecarga sensorial após exposição repentina a ruídos altos.');
```

BANCO DE DADOS RELACIONAL

```sql
INSERT INTO configuracoes_usuario (usuario_id, modo_alto_contraste, notificacoes_ativas) VALUES
(1, TRUE, TRUE),
(2, FALSE, TRUE);

INSERT INTO notificacoes (usuario_id, mensagem) VALUES
(1, 'Lembrete: Registre seu nível de conforto sensorial de hoje.');

INSERT INTO logs_auditoria (acao, detalhes, nivel_severidade) VALUES
('INIT_DB', 'Banco de dados relacional completo inicializado com sucesso para o SensivApp.', 'INFO
```

CAMADA DE APRESENTAÇÃO

DOCKERFILE

```dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

NGINX.CONF

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
    }
}
```

LÓGICA DO FRONT-END

Renderiza o formulário de diário de sintomas e consome os endpoints da API de forma assíncrona.

LÓGICA DO FRONT-END

Nossa interface utiliza JavaScript com funções assíncronas (fetch) para se comunicar com a API sem recarregar a página. Ela consome as rotas do back-end para listar dinamicamente os registros salvos no banco e envia novos inputs preenchidos pelo usuário através de requisições POST.

INDEX.HTML

```
<!DOCTYPE html>
<html lang="pt-BR">
<body>
<div class="container">
    <h2>SensivApp - Diário de Sintomas</h2>
    <form id="formSintoma">
        <label for="intensidade">Intensidade do Sintoma:</label>
        <select id="intensidade" required>
            <option value="Leve">Leve</option>
            <option value="Moderado">Moderado</option>
            <option value="Intenso">Intenso</option>
        </select>

        <label for="descricao">Descrição / Detalhes:</label>
        <textarea id="descricao" rows="3" placeholder="Como você está se sentindo hoje?" required></textarea>

        <button type="submit">Salvar Registro</button>
    </form>

    <h3>Registros Salvos no Banco:</h3>
    <div id="listaSintomas">Carregando...</div>
</div>

<script>
    const API_URL = "";

    function getDotClass(intensidade) {
        const val = intensidade.toLowerCase();
        if (val.includes('leve')) return 'dot-leve';
        if (val.includes('moderado')) return 'dot-moderado';
```

INDEX.HTML

```
31          if (val.includes('intenso')) return 'dot-intenso';
32          return 'dot-leve';
33      }
34
35      async function carregarSintomas() {
36          try {
37              const res = await fetch(`${API_URL}/sintomas/`);
38              const data = await res.json();
39              const div = document.getElementById("listaSintomas");
40              if (data.length === 0) {
41                  div.innerHTML = "<p>Nenhum registro encontrado.</p>";
42                  return;
43              }
44              div.innerHTML = data.map(s => `
45                  <div class="sintoma-item">
46                      <div class="badge-container">
47                          <span class="dot ${getDotClass(s.intensidade)}"></span>
48                          <span><strong>Intensidade:</strong> ${s.intensidade}</span>
49                      </div><br>
50                      <span style="display: block; margin-top: 6px;"><strong>Descrição:</strong> ${s.descricao}</span>
51                  </div>
52              `).join("");
53          } catch (e) {
54              document.getElementById("listaSintomas").innerHTML = "<p style='color:#ff6961;'>Erro ao conectar com a API.</p>";
55          }
56      }
```

INDEX.HTML

```
        }
    }

    document.getElementById("formSintoma").addEventListener("submit", async (e) => {
        e.preventDefault();
        const intensidade = document.getElementById("intensidade").value;
        const descricao = document.getElementById("descricao").value;

        await fetch(`${API_URL}/sintomas/`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ intensidade, descricao })
        });

        document.getElementById("descricao").value = "";
        carregarSintomas();
    });

    carregarSintomas();
</script>
</body>
</html>
```

STYLE.CSS

```css
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: #f7f5fa;
    color: #4a4556;
    margin: 0;
    padding: 30px;
}
.container {
    max-width: 600px;
    margin: auto;
    background: #ffffff;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 4px 12px rgba(177, 156, 217, 0.15);
    border: 1px solid #f0eaf8;
}
h2, h3 {
    color: #857494;
}
label {
    font-weight: 600;
    display: block;
    margin-top: 15px;
    color: #796e8b;
}
select, textarea {
    width: 100%;
    padding: 12px;
    margin-top: 6px;
    border: 1px solid #dcd6e8;
```

STYLE.CSS

```css
select:focus, textarea:focus {
    outline: none;
    border-color: #b19cd9;
    box-shadow: 0 0 0 3px rgba(177, 156, 217, 0.2);
}
button {
    width: 100%;
    padding: 12px;
    margin-top: 20px;
    background: #b19cd9;
    color: #ffffff;
    border: none;
    border-radius: 6px;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.2s;
}
button:hover {
    background: #9f86c9;
}
.sintoma-item {
    background: #fbf9fd;
    border-left: 4px solid #b19cd9;
    padding: 14px;
    margin-top: 12px;
    border-radius: 6px;
    box-shadow: 0 1px 3px rgba(0,0,0,0.02);
}
```

CAMADA BACKEND

DOCKERFILE

```dockerfile
FROM python:3.10-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    libpq-dev gcc \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

REQUIREMENTS.TXT

sensivapp-docker > api > requirements.txt

```
1  fastapi==0.110.0
2  uvicorn==0.28.0
3  sqlalchemy==2.0.28
4  psycopg2-binary==2.9.9
5  pydantic==2.6.4
```

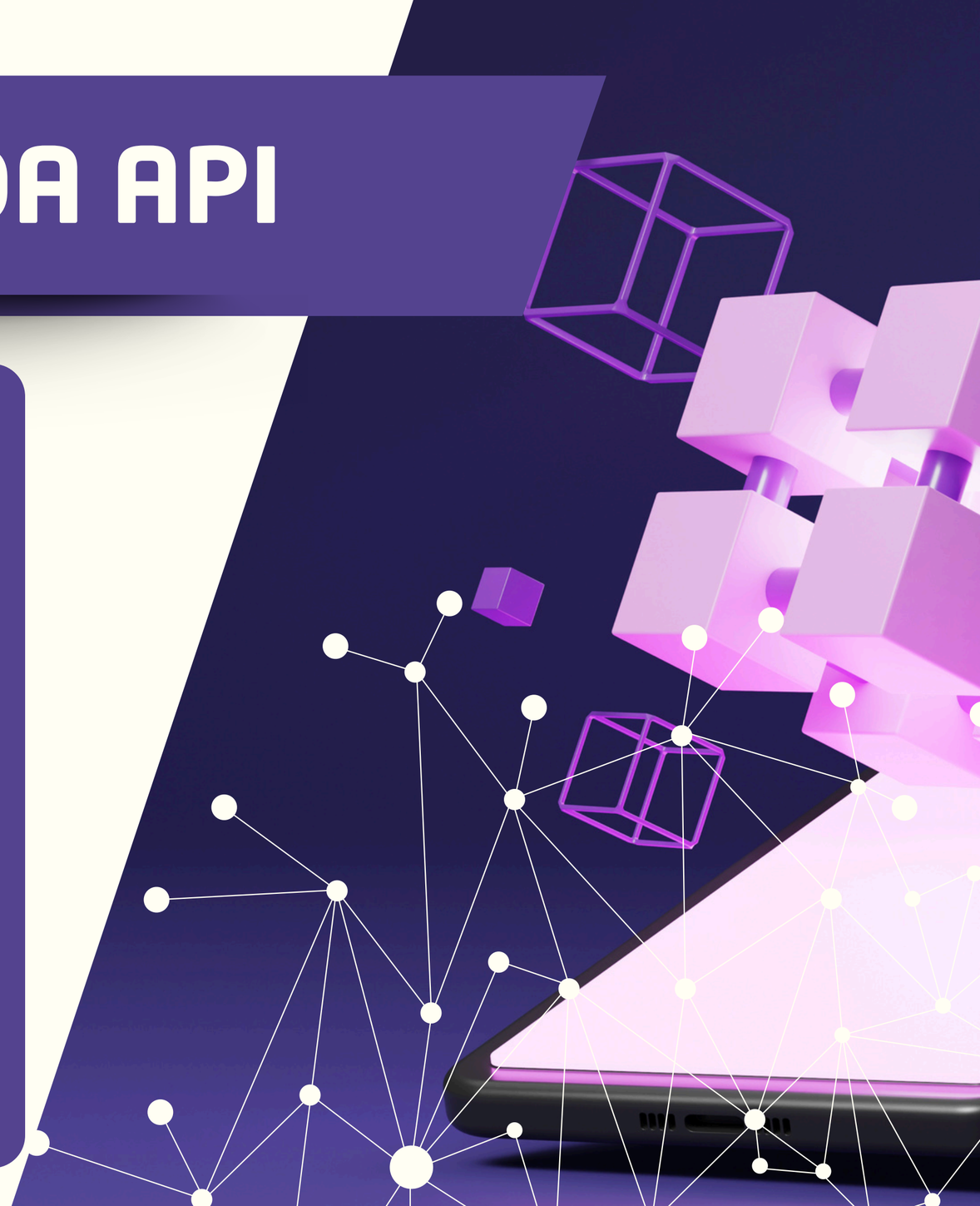
LÓGICA DE NEGÓCIOS DA API

O back-end foi estruturado em Python com FastAPI para garantir alta performance e assincronismo.

O código mapeia o modelo de dados de sintomas através do SQLAlchemy e expõe as rotas:

- uma rota GET para listar todos os registros armazenados

- uma rota POST que valida e persiste novos dados enviados pelo front-end com total segurança.



MAIN.PY

```python
import os
from fastapi import FastAPI, Depends
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from sqlalchemy import create_engine, Column, Integer, String, Text
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://sensiv:password@db:5432/sensivapp_db")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class SintomaModel(Base):
    __tablename__ = "sintomas"

    id = Column(Integer, primary_key=True, index=True)
    intensidade = Column(String(50), nullable=False)
    descricao = Column(Text, nullable=True)

Base.metadata.create_all(bind=engine)

app = FastAPI(title="SensivApp - Diário de Sintomas API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
```

MAIN.PY

```python
    allow_headers=["*"],
)

class SintomaSchema(BaseModel):
    intensidade: str
    descricao: str

    class Config:
        from_attributes = True

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/")
def read_root():
    return {"message": "API do SensivApp rodando com sucesso no cluster Swarm!"}

@app.post("/sintomas/", response_model=SintomaSchema)
def criar_sintoma(sintoma: SintomaSchema, db: Session = Depends(get_db)):
    novo_sintoma = SintomaModel(intensidade=sintoma.intensidade, descricao=sintoma.descricao)
    db.add(novo_sintoma)
    db.commit()
    db.refresh(novo_sintoma)
    return novo_sintoma
```

MAIN.PY

```
59
60  @app.get("/sintomas/")
61  def listar_sintomas(db: Session = Depends(get_db)):
62      return db.query(SintomaModel).all()
```

INFRAESTRUTURA, CLUSTER E DOCKER-IN-DOCKER

DOCKER-IN-DOCKER E CLUSTER

Cria os nós isolados (manager, worker1, worker2) via containers e inicializa o cluster Swarm.

DOCKER-IN-DOCKER

```
vannapaulo →/workspaces/sensivapp-docker/sensivapp-docker (main) $ docker network create --driver bridge swarm-net

er run -d --name node-manager --hostname node-manager --network swarm-net --privileged -v /var/run/docker.sock:/var/run/docker.sock docker:dind
er run -d --name node-worker1 --hostname node-worker1 --network swarm-net --privileged -v /var/run/docker.sock:/var/run/docker.sock docker:dind
er run -d --name node-worker2 --hostname node-worker2 --network swarm-net --privileged -v /var/run/docker.sock:/var/run/docker.sock docker:dind
```

```
@giovannapaulo →/workspaces/sensivapp-docker/sensivapp-docker (main) $ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS            NAMES
5ebcaffee6df   docker:dind    "dockerd-entrypoint.…"   7 seconds ago    Up 6 seconds    2375-2376/tcp    node-worker2
20dde8256b8a   docker:dind    "dockerd-entrypoint.…"   7 seconds ago    Up 7 seconds    2375-2376/tcp    node-worker1
21d1a404aa10   docker:dind    "dockerd-entrypoint.…"   7 seconds ago    Up 7 seconds    2375-2376/tcp    node-manager
```

INICIANDO O SWARM

```
@giovannapaulo ➜/workspaces/sensivapp-docker/sensivapp-docker (main) $ docker exec -it node-manager docker swarm init --advertise-addr $(docker inspect -f '{{
range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' node-manager)

JOIN_TOKEN=$(docker exec node-manager docker swarm join-token worker -q)
MANAGER_IP=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' node-manager)

docker exec -it node-worker1 docker swarm join --token $JOIN_TOKEN $MANAGER_IP:2377
docker exec -it node-worker2 docker swarm join --token $JOIN_TOKEN $MANAGER_IP:2377
```

```
@giovannapaulo ➜/workspaces/sensivapp-docker/sensivapp-docker (main) $ docker exec -it node-manager docker node ls
ID                            HOSTNAME       STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
68ytj8ngu7s230v9to58uzim9     5ebcaffee6df   Ready     Active                          29.9.3
ipn0tu3134frr9i7aiwvkf3t6     20dde8256b8a   Ready     Active                          29.9.3
p4jnv7ey491xuanat5amg1pqw *   21d1a404aa10   Ready     Active         Leader           29.9.3
```

INICIANDO SERVIÇOS

```
@giovannapaulo ➜/workspaces/sensivapp-docker/sensivapp-docker (main) $ docker cp docker-compose.yml node-manager:/tmp/docker-compose.yml
Successfully copied 1.19kB (transferred 3.07kB) to node-manager:/tmp/docker-compose.yml
```

```
docker exec -it node-manager docker stack deploy -c /docker-compose.yml sensivstack
Successfully copied 1.2kB (transferred 3.07kB) to node-manager:/docker-compose.yml
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network sensivstack_sensiv_net
Creating service sensivstack_visualizer
Creating service sensivstack_db
Creating service sensivstack_api
Creating service sensivstack_frontend
@giovannapaulo ➜/workspaces/sensivapp-docker/sensivapp-docker (main) $
```

STATUS DOS SERVIÇOS

```
● @giovannapaulo ➜/workspaces/sensivapp-docker/sensivapp-docker (main) $ docker exec -it node-manager docker stack services sensivstack
ID             NAME                    MODE         REPLICAS   IMAGE                                PORTS
eiuxghlo4w9x   sensivstack_api         replicated   3/3        sensiv-api:latest
wtidqoew7ytn   sensivstack_db          replicated   1/1        sensiv-db:latest
ybpcbtzvx4l7   sensivstack_frontend    replicated   2/2        sensiv-frontend:latest               *:3000->80/tcp
2iv6994pbu25   sensivstack_visualizer  replicated   1/1        dockersamples/visualizer:stable      *:8080->8080/tcp
```

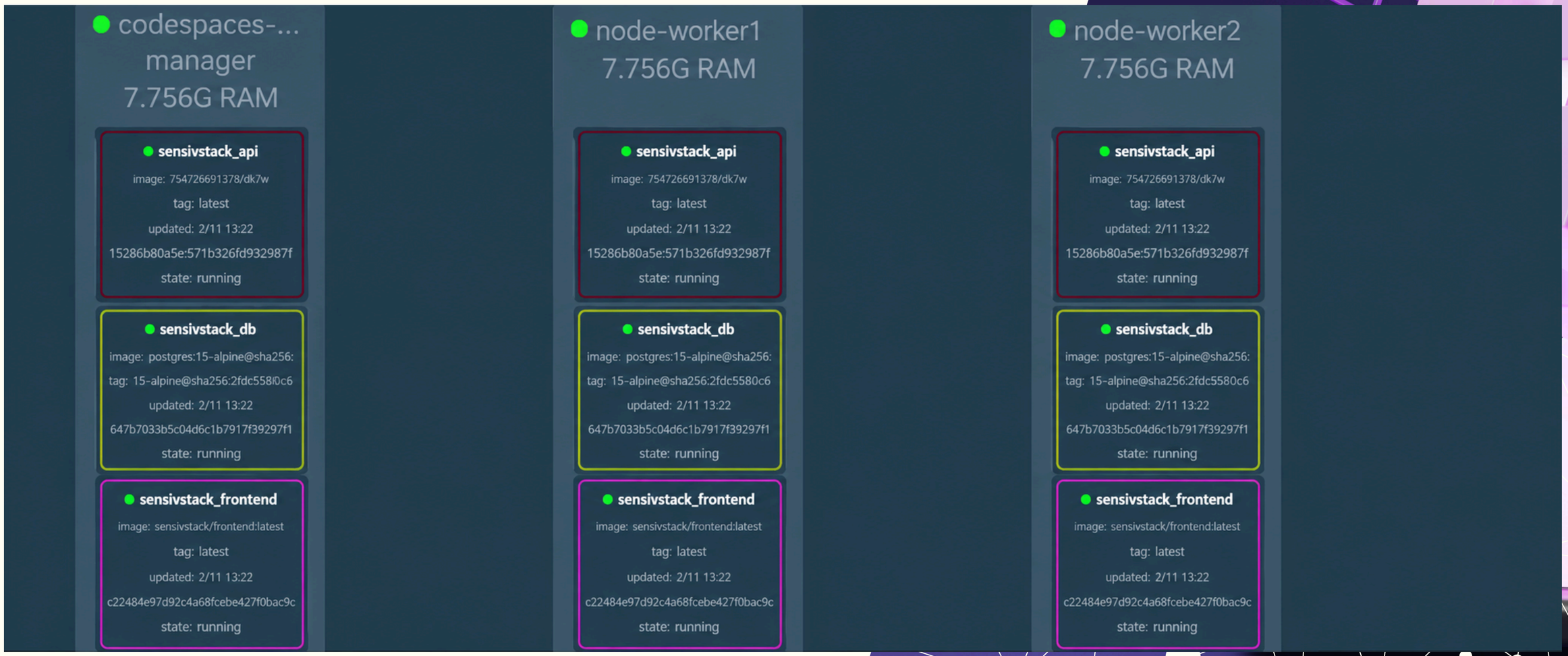
VISUALIZER E SIMULAÇÕES NO DOCKER

DOCKER VISUALIZER

Para validar visualmente a distribuição do cluster em tempo real, integramos o Docker Visualizer como ferramenta de auditoria e monitoramento gráfico.



DOCKER VISUALIZER

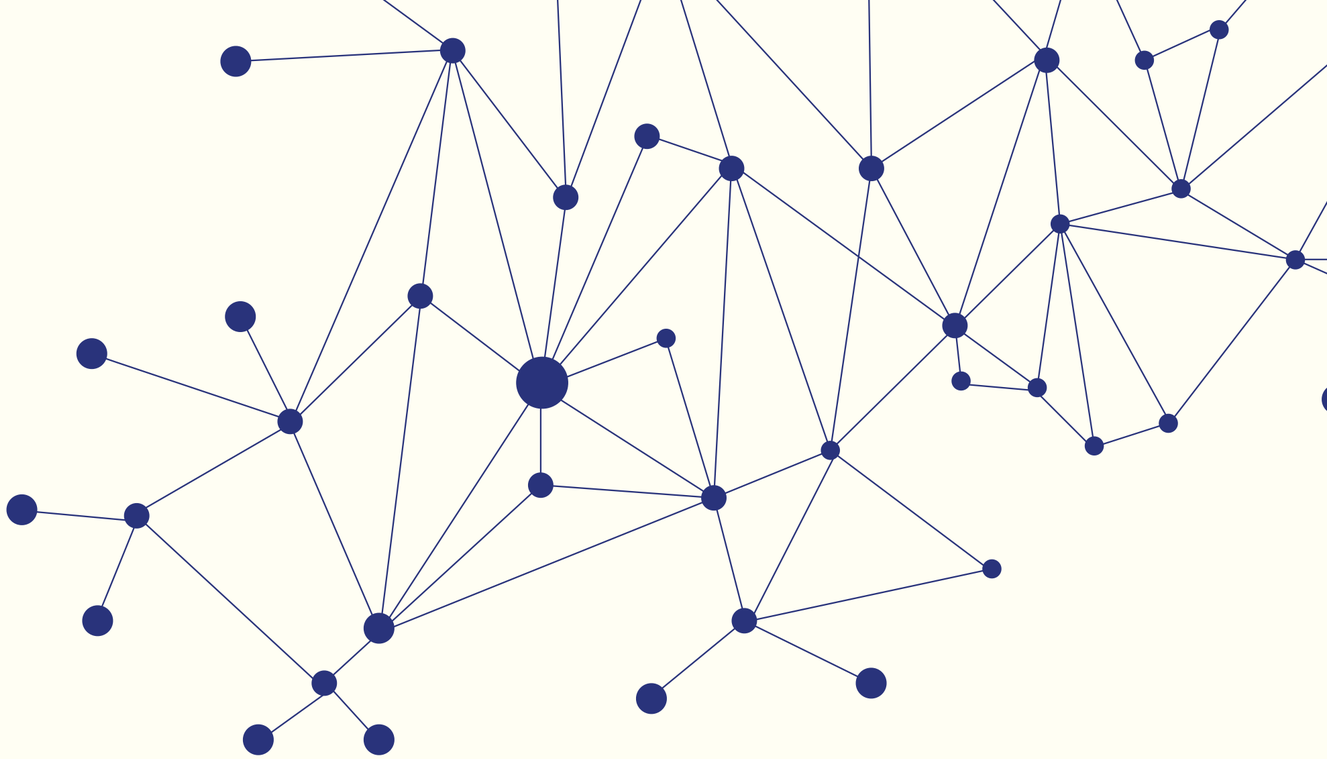


SIMULAÇÃO (ALTA DISPONIBILIDADE E FALHA)

CONCLUSÃO

No SensivApp, o uso do Docker e do Docker Swarm impacta diretamente a aplicação ao garantir alta disponibilidade e proteção para dados sensíveis de saúde. Com o isolamento de microsserviços, uma falha na interface não afeta o armazenamento dos registros no banco de dados, enquanto o sistema de auto-healing e o balanceamento de carga asseguram que a plataforma permaneça sempre ativa, estável e segura para o acompanhamento dos usuários.

Obrigado pela atenção!



SENSIVAPP